LAB 1: 2D Matrix

[CO5]

Instructions for students:

- Complete the following methods based on Array & 2D Matrix.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.
- If you are using JAVA, then follow the **Java template.**
- If you are using PYTHON, then follow the **Python template**.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAYS.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

[Make changes to the Sample Inputs and check whether your program works correctly]

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Assignment Tasks: 4
Total Marks: 20
Converted Marks: 4

Intro Lab Tasks

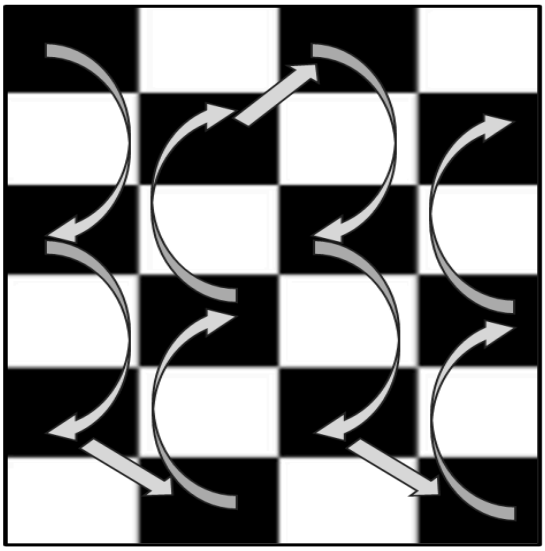
The following tasks are here to give you a brief idea about basic 2D Matrix operations. Once you're comfortable with basic coding of 2D Matrix then you can move on to the Main Lab Tasks. No coding template for these intro Lab Tasks.

I. Print a given 2D matrix row wise.
II. Print a given 2D matrix column wise.
III. Explain the difference between I and II to yourself.
IV. Find the absolute difference between the sum of primary diagonal and secondary diagonal of a given 2D matrix.
V. Transpose a given 2D Matrix.

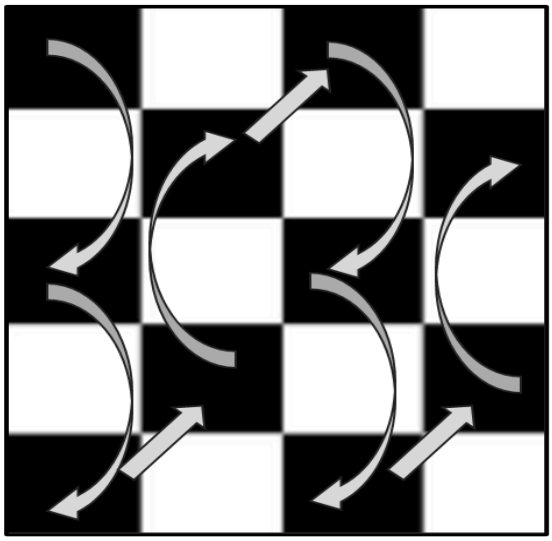
Main Lab Tasks [NO NEED TO SUBMIT]

1. Zigzag Walk:

As a child, you often played this game while walking on a tiled floor. You walked avoiding a certain color, for example white tiles (it almost felt like if you stepped on a white tile, you would die!). Now you are in a room of m x n dimension. The room has m*n black and white tiles. You step on the black tiles only. Your movement is like this:



OR



Now suppose you are given a 2D array which resembles the tiled floor. Each tile has a number. Can you write a method that will print
your walking sequence on the floor?

Hint 1: The first tile of the room is always black.
Hint 2: Look out for the number of rows in the matrix Notice the transition from column 0 to column 1 in the above figures

<table>
<tr><th>Sample Given Matrix</th><th>Sample Output</th></tr>
<tr><td>
<table>
<tr><td>3</td><td>8</td><td>4</td><td>6</td><td>1</td></tr>
<tr><td>7</td><td>2</td><td>1</td><td>9</td><td>3</td></tr>
<tr><td>9</td><td>0</td><td>7</td><td>5</td><td>8</td></tr>
<tr><td>2</td><td>1</td><td>3</td><td>4</td><td>0</td></tr>
<tr><td>1</td><td>4</td><td>2</td><td>8</td><td>6</td></tr>
</table>
</td><td>3 9 1
1 2
4 7 2
4 9
1 8 6</td></tr>
<tr><td>
<table>
<tr><td>3</td><td>8</td><td>4</td><td>6</td><td>1</td></tr>
<tr><td>7</td><td>2</td><td>1</td><td>9</td><td>3</td></tr>
<tr><td>9</td><td>0</td><td>7</td><td>5</td><td>8</td></tr>
<tr><td>2</td><td>1</td><td>3</td><td>4</td><td>0</td></tr>
</table>
</td><td>3 9
1 2
4 7
4 9
1 8</td></tr>
</table>

2. Decryption Process:

Suppose you're working as a cryptographer for a secret intelligence agency. You are given an encrypted matrix as an input. You have to decrypt it after some processing of the given matrix.

To decrypt this message efficiently, you have a task to construct a method called **decrypt_matrix(matrix)** which takes an encrypted matrix as input and returns a decrypted linear array. The process of finding the decrypted linear array is given below:

You have to find out the column-wise summations for each column and store the difference of subsequent column-wise summations in a new linear array.

<table>
<tr><th>Sample Given Matrix</th><th>Sample Output</th><th>Explanation</th></tr>
<tr>
<td>
<table>
<tr><td>1</td><td>3</td><td>1</td></tr>
<tr><td>6</td><td>4</td><td>2</td></tr>
<tr><td>5</td><td>1</td><td>7</td></tr>
<tr><td>9</td><td>3</td><td>3</td></tr>
<tr><td>8</td><td>5</td><td>4</td></tr>
</table>
</td>
<td>
<table>
<tr><td>-13</td><td>1</td></tr>
</table>
</td>
<td>
Sum of 0th column = 29
Sum of 1st column = 16
Sum of 2nd column = 17

Therefore, the size of the resulting array is 2 and the array is:
<table>
<tr><td>16-29 = -13</td><td>17-16 = 1</td></tr>
</table>
</td>
</tr>
</table>

<u>Assignment Tasks</u> [NEED TO SUBMIT]

Won't be explained in lab class

1. Row Rotation Policy of Your Classroom [5 marks]

You are no longer permitted to choose your own seat on the new campus of your university. You must abide by the new regulations, which state that if you sit in the first row for the first week, you must shift to the second row for Week2, the third row for the Week3, and so on. In your classroom, there are a total of 6 rows and 5 columns. Your friend "AA" wants to know from you that on the upcoming exam week in which row he will be in. Your task is to implement a function **`row_rotation(exam_week, seat_status)`** that takes the exam_week and a 2D array of current seat status as input and returns the row number in which your friend **"AA"** will be seated and print the seat status for that week.

<u>NOTE</u>: If the shape of the given matrix is MxN then the space complexity of your solution must not be greater than O(N).

Sample User Input & Given Matrix	**Output**
User Input: exam_week = 3 **Given Matrix:** **current_seat_status =** | A | B | C | D | E | | F | G | H | I | J | | K | L | M | N | O | | P | Q | R | S | T | | U | V | W | X | Y | **| Z | AA | BB | CC | DD |**	| U | V | W | X | Y | **| Z | AA | BB | CC | DD |** | A | B | C | D | E | | F | G | H | I | J | | K | L | M | N | O | | P | Q | R | S | T | Your friend AA will be on row 2
<u>Explanation</u>: According to the example the exam is 3 weeks away from the current seat status. So, during the exam day, the row containing **AA** will move **2 times** (basically the whole matrix will rotate downward vertically).	

2. Matrix Compression [5 marks]

Complete the function **compress_matrix** that takes a 2D array as a parameter and return a new compressed 2D array. In the given array the number of row and column will always be even. **Compressing a matrix means grouping elements in 2x2 blocks and sums the elements within each block. Check the sample input output for further clarification.**

Hint: Generally the block consists of the (i,j), (i+1,j), (i,j+1) and (i+1, j+1) elements for 2x2 blocks.

You cannot use any built-in function except len() and range(). You can use the np variable to create an array.

Python Notation	Java Notation

import numpy as np
def compress_matrix (mat):
    # To Do
	public int[ ][ ] compress_matrix (int[] [] mat) {
	    // To Do
	}

Sample Input array	All Box (No need to create these arrays)	Returned Array	Explanation
[[1, 2, 3, 4], [5, 6, 7, 8], [1, 3, 5, 2], [-2, 0, 6,-3]]	[[1, 2], [[3, 4], [5, 6]] [[7, 8]] [[1, 3], [[5, 2], [-2, 0]] [[6, -3]]	[[14, 22], [2, 10]]	[[1+2+5+6, 3+4+7+8], [1+3+-2+0, 5+2+6+-3]]

3. Game Arena [5 marks]

Suppose you and your friends are in the world of '*Alice in Borderland*' where you decided to take part in a game and entered the *game arena*. As a team, you need to gain **at least 10 points** in order to keep surviving in the borderland. Otherwise, you will be out of the game and your team will be banished for good. Now, the arena has a 2D array like structure where players of a team are given certain positions with values that are **multiples of 50**. By staying in these positions, every player can gain points from the cells above, below, left and right (not diagonally) only if those cells **contain 2** [The cells containing 1s and 0s are to be avoided]. For each player, add from these cells containing 2s to your total points for the team to keep on surviving in borderland. **Be careful about corner cases.**

Your task is to write a method which tells us whether your team is out or has survived the game.

Sample Given Matrix 1	Sample Output 1
| 0 | 2 | 2 | 0 | | **50** | 1 | 2 | 0 | | 2 | 2 | 2 | 0 | | 1 | **100** | 2 | 0 |	Points Gained: 6. Your team is out.

Explanation:
Player with value 50 has 2 in the cell below him (1 cell). Player with value 100 has 2 in the cell above and in the right cell (2 cells). So in total, they got (1+2)*2 = 6 points which was not enough to survive the game.

Sample Given Matrix 2	Sample Output 2
| 0 | 2 | 2 | 0 | 2 | | 1 | **50** | 2 | 1 | **100** | | 2 | 2 | 2 | 0 | 2 | | 0 | **200** | 2 | 0 | 0 |	Points Gained: 14. Your team has survived the game.

Explanation:
Player with value 50 has 2 in the cell above, cell below and the right cell (3 cells). Player with value 100 has 2 in the cell above and the cell below (2 cells). Player with value 200 has 2 in the above cell and right cell (2 cells). So in total, they gained (3+2+2)*2 = 14 points and survived the game.

Note: For the cell with value 2 that is common between 2 players in position (2,1), both gained 2 points each, so it's not like if one player already added those 2 points, another player cannot. In the Given Matrix 2, Player with value 50 and player with value 200 are sharing a common 2 but both of them got points.

4. Rotate Secret [5 marks]

Your friend sent you a secret message scattered across a square board. The letters are scrambled, making the message unreadable. To recover it, you must rotate the concentric layers of the board clockwise, starting from the innermost layer and moving outward. You are given a square N × N matrix of characters, where both N (rows and columns) are even. Rotate each layer clockwise starting from the innermost layer, the innermost rotates once, and each outer layer rotates one time more than the one inside it. After performing all rotations, the hidden message will be revealed.

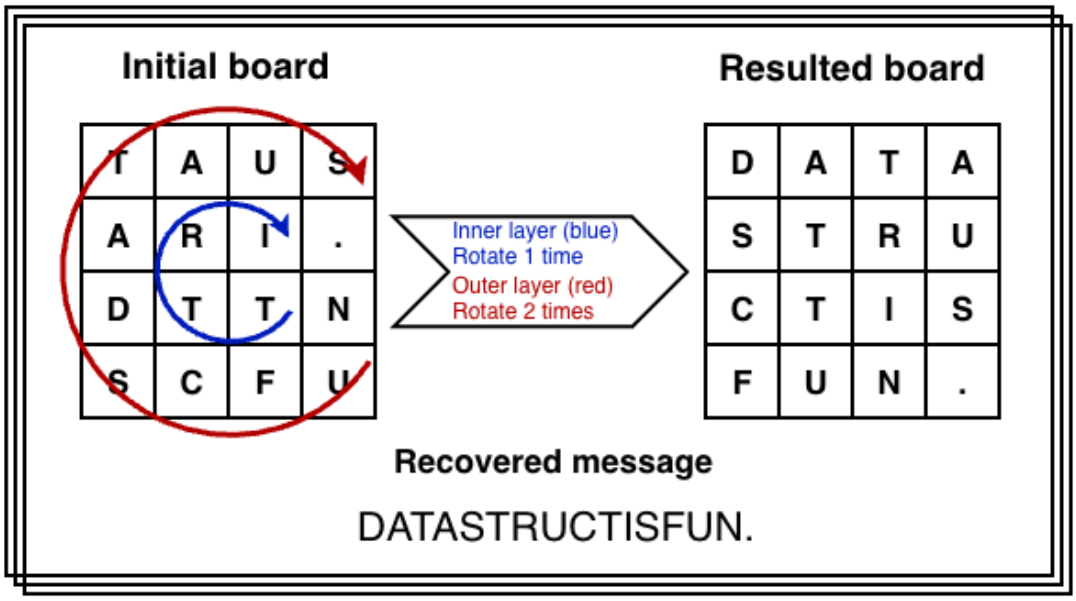
Write a method/function named **rotateSecret()**, which rotate the matrix clockwise :

- Innermost layer rotate 1 time
- Next outer layer rotate 2 times
- Next outer rotate 3 times, and so on.

After all successful rotations, print the recovered message.

Space Complexity of the solution must be O(1)

[can't create new arrays]



Sample Given Matrix 1	After Processing	Output

board = {
{'T','A','U','S'},
{'A','R','I','.'},
{'D','T','T','N'},
{'S','C','F','U'}
}
	board = {
	{'D','A','T','A'},
	{'S','T','R','U'},
	{'C','T','I','S'},
	{'F','U','N','.'}
	};
		DATASTRUCTISFUN.

Explanation		
1 clockwise rotation on the innermost (2×2) layer, and 2 clockwise rotations on the outer (4×4) layer, each letter shifts to its correct position after these rotations. Then the message from the board is printed.		

Sample Given Matrix 2	After Processing	Output

board = {
{'O','R','I','R','N','P'},
{'G','S','A','A','L','R'},
{'L','M','N','O','N','Y'},
{'A','H','U','O','O','P'},
{'T','F','C','T','H','S'},
{'E','D','Y','O','C','K'}
}
	{
	{'A','L','G','O','R','I'},
	{'T','H','M','S','A','R'},
	{'E','F','U','N','A','N'},
	{'D','C','O','O','L','P'},
	{'Y','T','H','O','N','R'},
	{'O','C','K','S','P','Y'}
	}
		ALGORITHMSAREFUNANDCO
		OLPYTHONROCKSPY

Explanation

1 clockwise rotation on the innermost (2×2) layer, and
2 clockwise rotations on the outer (4×4) layer,
3 clockwise rotations on the outer (6×6) layer,
each letter shifts to its correct position after these rotations. Then the message from the board is printed.

LAB 2: Node & LinkedList

[CO1]

Instructions for students:

- You may use Java / Python to complete the tasks.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN THE LINKED LIST YOU'RE CREATING.**
- **IF THE QUESTION ALLOWS YOU TO CREATE OTHER DATA STRUCTURES THEN YOU CAN.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 2 Assignment Tasks: 4
Total Marks: 20
Converted Marks: 4

Intro Node Tasks

This is an intro task & there isn't any driver code for this

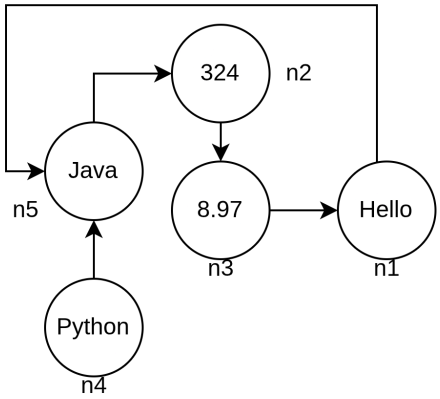
- ❖ For java, create a separate folder for this task and follow the instructions.
- ❖ For Python, create a separate colab/ipynb/py file and follow the instructions.

➔ Design a Node class.

- Declare two instance variables, one called **elem** (Object data type for java) and another one called **next** (Node datatype).
- Write a constructor that has only one parameter (Object data type for java) and assigns the parameter value to the **elem** instance variable.

➔ Design a Tester class (for java) / Design another code cell (for python)

- Create 5 different objects of Node class. Assign values as shown in the illustration.
- Variable names should be: **n1, n2, n3, n4, n5**.
- Connect the 5 nodes as shown in the illustration.



Now, execute the lines given below and try to understand the output by simulating them using pen & paper

If there are errors, try to figure out why that error occurred and what can be done about it.

Note: Java & Python output won't always be the same.

JAVA	PYTHON
`System.out.println( n1 );` `System.out.println( n3.next );`	`print( n1 )` `print( n3.next )`
`System.out.println( n3.next.elem );`	`print( n3.next.elem )`
`Node x = n4.next;` `System.out.println( n1.elem + x.elem );`	`x = n3.next` `print( x.elem + n4.elem )`
`x.next = n3;` `System.out.println(n2.next.next + n5.next);`	`x.next = n2` `print(n2.next.next + n5.next)`
`n5 = null;` `System.out.println( n4.next.elem );`	`n5 = None` `print( n4.next.elem )`
`x.next.next = null;` `n3.next.elem = 321;`	`x.next.next = None` `n2.next.elem = 7.98`
`n4.next = 532;` `System.out.println(n4.next.elem);`	`n4.next = 532` `print(n4.next.elem)`

Now let's design a basic Linked List Class.
Please use the Java Intro LinkedList Template or Python Intro LinkedList Template .

Intro Linked List: Design Linked list class

Node class is already given, use that from the template.

1. Complete the **append()** method inside the LinkedList class. This method takes a value as a parameter and inserts it into the back of the linked list.
    - Insert a new node at the end of the list.
    - If the list is empty, the new node becomes the head.

2. Complete the **printList()** method, which Prints all elements starting from head.
    - Traverse until the end of the list.

3. Complete the **prepend()** method inside the LinkedList class. This method takes a value as a parameter and inserts it into the front of the linked list.
    - Update the head reference properly.

4. Complete the **nodeAt()** method that returns the node at the given index (0 based).
    - If the index is invalid, return null.
    - Do NOT print inside this method.

5. Complete the **removeFirst()** method inside the LinkedList class. This should remove the first node from the linked list.
    - If the list is empty, do nothing.

6. Complete the **removeLast()** method inside the LinkedList class. This should remove the last node from the linked list.
    - If the list has only one node, head should become null.
    - If the list is empty, do nothing.

Congratulations! You have successfully designed a LinkedList class.

However, in the upcoming problem solving tasks, you will not be allowed to use this class. Instead, you will be given the head of a linked list and asked to complete specific tasks. If you need any additional helper methods, you must implement them yourself.

Now we can move on to actual problem solving using nodes/linkedlist. Please use the Java LabTask Template or Python LabTask Template

Main Lab Tasks [No Need to Submit]

1. Assemble Conga Line

Have you ever heard the term conga line? Basically, it's a carnival dance where the dancers form a long line. Everyone holds the waist of the person in front of them and their waists are held in turn by the person to their rear, excepting only those in the front and the back. It kind of looks like this.

By now, you can quite understand the suitable data structure to represent a conga line. Now you are the choreographer of the Conga Dance in a Summer Festival. You wish to arrange the conga line **ascending** age wise and tell the participants to stand in a line likewise. Now as technical you are, can you write a method that will take the conga line and return True if everyone stands according to your instruction. Otherwise returns False.

Sample Input	Sample Returned Result
10 → 15 → 34 → 41 → 56 → 72	True
10 → 15 → 44 → 41 → 56 → 72	False

2. Word Decoder

Suppose, you have been hired as an cyber security expert in an organization. A mysterious code letter has been discovered by your team, your task is to decode the letter. After lots of research you found a pattern to solve the problem. Problem details are given below:

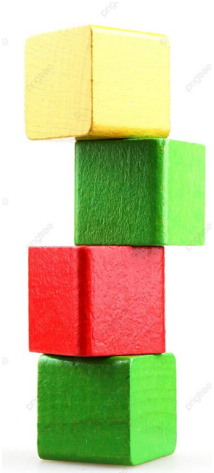
- For each encoded word a linked list will be given, where each node will carry one letter as an element.
- For the decoded word, the letters are at the multiples of $(13 \% length\ of\ linkedlist)$ position **[0 indexed].** For example, if the length of the given linked list is 10, then 13 % 10 = 3 and the letters are at positions which are multiples of 3 (i.e. at position 3, 6, 9)
- However, the letters are stored in the given linked list in opposite order. Thus the decoded word has to be reversed.
- After decoding, your program should return a dummy headed singly linked list containing the decoded word.

Sample Input	Sample Output
B→M→D→T→N→O→A→P→S→C	None → C→A→T **Explanation:** Length of the list= 10 & 13%10=3 The letters are T, A, C at 3, 6, 9 positions respectively **[0 indexed].** The letters are reversed and the resulting linked list is None → C→A→T
Z→O→T→N→X	None → N **Explanation:** Length of the list= 5 & 13%5=3. The letter is N at position 3 **[0 indexed].** The letters are reversed and the resulting linked list is None → N

Assignment Tasks [Need to Submit]

1. Building Blocks

Your twin and you are under an experiment where the amount of thinking similarities you two have is being observed. As per the experiment, you are given a certain number of building blocks of different colors and are told to make a building using those blocks in two different rooms.



After the buildings are finished, the observers check whether the two buildings are the same based on the block colors. Now, you are the tech guy of that team and you are instructed to write a program that will output "Similar" or "Not Similar" given the two buildings. For fun, you decided to represent those buildings as a linked list!

NB: Red means a red block
Blue means a blue block
Yellow means a yellow block
Green means a green block.

Sample Input	Sample Output
building_1 = Red→ Green→ Yellow→ Red→ Blue→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green	Similar
building_1 = Red→ Green→ Yellow→ Red→ Yellow→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green	Not Similar
building_1 = Red→ Green→ Yellow→ Red→ Blue→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green→ Blue	Not Similar

2. Organize Books

You are a system engineer at a university library that maintains its collection of books digitally using a singly linked list. Each node in the linked list represents a book title as a string stored in the catalog.
To improve book recommendations, the library recently collected popularity scores from student surveys. Each score represents how popular a book is, a higher score means greater popularity. Your task is to reorganize the linked list so that books are arranged in descending order of popularity. If multiple books have the same popularity score, they must retain their original order.

Write a function named **organizeBooks(Node head, int[] popularity)**, which will take the **head** of the linked list and an integer array containing the **popularity** score. After organizing the list, **return the head of the modified list.**

- You can not create or take a new linked list, array or any other data structure.

Sample Given LL and Array Example1	**Output**
Dune ➙ IT ➙ Coraline ➙ Inferno ➙ Twilight [8, 10, 5, 10, 6]	IT ➙ Inferno ➙ Dune ➙ Twilight ➙ Coraline
Explanation Example1: In the given list, Dune has a score of 8, IT has 10, Coraline has 5, Inferno has 10, and Twilight has 6. The list is reorganized so that books with higher popularity appear first. Therefore, IT and Inferno, both with a score of 10, come to the front. Next is Dune with a score of 8, followed by Twilight with 6, and finally Coraline with 5.	
Sample Given LL and Array Example2	**Output2**
Hamlet ➙ Persuasion ➙ It ➙ Dracula ➙ Beloved [7, 9, 9, 6, 7]	Persuasion ➙ It ➙ Hamlet ➙ Beloved ➙ Dracula
Sample Given LL and Array Example3	**Output3**
Matilda ➙ Franny ➙ Foundation ➙ Carrie ➙ Misery [5, 8, 8, 10, 6]	Carrie ➙ Franny ➙ Foundation ➙ Misery ➙ Matilda

3. Alternate Merge

You are given two singly non-dummy headed linked lists. Your task is to write a function **alternate_merge(head1, head2)** that takes the heads of the two linked lists and returns the head of a modified linked list with all the elements of the two lists in alternate order. It is guaranteed that alternate placement is always possible. Your resulting linked list will always start with the head of linked list 1.

Input	Output
List1: 1 → 2 → 6 → 8 → 11 → None List2: 5 → 7 → 3 → 9 → 4 → None	1 → 5 → 2 → 7 → 6 → 3 → 8 → 9 → 11 → 4 → None
List1: 5 → 3 → 2 → -4 → None List2: -4 → -6 → 1 → None	5 → -4 → 3 → -6 → 2 → 1 → -4 → None
List1: 4 → 2 → -2 → -4 → None List2: 8 → 6 → 5 → -3 → None	4 → 8 → 2 → 6 → -2 → 5 → -4 → -3 → None

NOTE: The space complexity of the solution must be O(1). Which means, You're NOT ALLOWED to create any new linked list for this task.

4. ID Generator

Write a function **idGenerator()** that will take three non-dummy headed linear singly linked lists containing only digits from 0-9 and **generate another singly linked list** with 8 digit student ID from it [The student ID contains only digits from 0 to 9].

The first linked list will have the first four digits of the student id in reverse order. The remaining four digits can be obtained by adding the elements of the second linked list **from the beginning** with the elements of the third linked list **from the beginning**.

If the summation is >= 10, then you have to mod the summation by 10 and insert the result in the output linked list. For example, in sample input 2, the last element of the second list is 7 and the last element of the third linked list is 8. So after adding them we get 7+8=15>10. So the digit we will insert as an element of the Student ID list, will be 15%10=5.

Sample Input 1:	**Sample Input 2:**
0 → 3 → 2 → 2	0 → 3 → 9 → 1
5 → 2 → 2 → 1	3 → 6 → 5 → 7
4 → 3 → 2 → 1	2 → 4 → 3 → 8
Sample output 1:	**Sample output 2:**
2 → 2 → 3 → 0 → 9 → 5 → 4 → 2	1 → 9 → 3 → 0 → 5 → 0 → 8 → 5

You Will Have To Submit Lab 2 and Lab 3 as a single file after next week. Basically, There will be only 1 combined submission for Lab 2 & Lab 3.

LAB 3: Doubly LinkedList

[CO1]

Instructions for students:

- You may use Java / Python to complete the tasks.
- If you are using **JAVA**, then follow the **Java Template**.
- If you are using **PYTHON**, then follow the **Python template**.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN THE LINKED LIST YOU'RE CREATING.**
- **IF ONLY THE QUESTION ALLOWS YOU TO CREATE OTHER DATA STRUCTURES, THEN YOU CAN.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 3 Assignment Tasks: 3
Total Marks: 15

Lab Task

In this part of the lab, we will play with Dummy Headed Doubly Circular Linked List. If you want to read about this type of linked list then check **this file.**

> **Note: Even though we're learning Dummy Headed Doubly Circular Linked List, the methods you'll be writing simulate a different data structure. Can you guess what that is? Hint: It's something you face in front of the elevator everyday.**

In the first part of this lab, you have to implement a waiting room management system in an emergency ward of a hospital. Your program will serve a patient on a **first-come, first-served basis.** The **simulation of WRM** can be found **here**.

Solve the above problem using a **Dummy Headed Doubly Circular Linked List.**

1. You need to have a **Patient** class so that you can create an instance of it (patient) by assigning id(integer), name (String), age (integer), and bloodGroup (String).
2. Write a **WRM** (waiting room management) class that will contain the following methods.
    a. **registerPatient(id, name, age, bloodgroup):** This method will register a patient into your system. The method will create a Patient type object with the information received as a parameter. It means this method will add a patient-type object to your linked list.
    b. **servePatient():** This method calls a patient to provide hospital service to him/her. In this method, you need to ensure that to serve the patient who was registered first. (serving means removing the patient from line)
    c. **cancelAll():** This method cancels all appointments of the patients so that the doctor can go to lunch.
    d. **canDoctorGoHome():** This method returns true if no one is waiting; otherwise, it returns false.
    e. **showAllPatient():** This method prints all IDs of the waiting patients in sequential order. It means the patient who got registered first will come first, and so on.
    f. **reverseTheLine():** This method reverses the patient line. It means the patient who got registered last will come first, and so on.

Important Hints:

1. In your program, there won't be any class called **"Node".** Instead, the **Patient class** will work as the Node class.
2. On the other hand, the **WRM** class will act as the Dummy Headed Doubly Circular Linked List, which will contain all the other functions/methods.

Assignment Tasks [Need to Submit]

If you need to use extra helper functions/methods feel free to create & use them. If you use them then make sure to submit those extra functions/methods as well. Here's the Java Template & Python Template

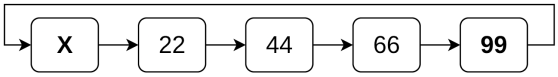
[continued from the previous assignment...]

5. Sum Odd Append

Write a method/function called **sumOddAppend().** The function will take only one parameter**,** referencing a **dummy-headed singly circular linked list**. The function/method removes all the nodes containing odd values while summing them up. Finally, a node containing the summation is inserted at the end.

NOTE:

- YOU **CAN** CREATE **ONLY 1 EXTRA NEW NODE** using the given Node Class.

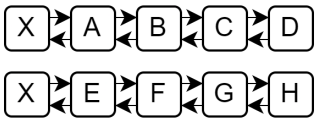
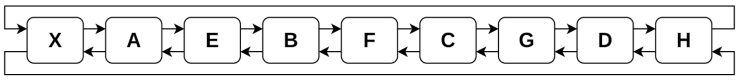
Given Dummy Headed Singly Circular Linked List	Expected Modified Linked List
	

6. Pair Join

Write a method/function called **pairJoin()**. The function takes two parameters, referencing **two dummy heads** of two different Dummy Headed Doubly Linked Lists. The method/function will modify the connections of the two given linked lists and combine them in pairs as shown in the example below. The method/function will not return anything because the first dummy head will naturally become the new head.

NOTE:

1. YOU **CANNOT** CREATE ANY NEW NODES.
2. YOU CAN ASSUME THE LENGTH OF THE GIVEN TWO LISTS WILL ALWAYS BE EQUAL.

Given Dummy Headed Doubly Linked List	Expected Modified Linked List
	

7. Range Move

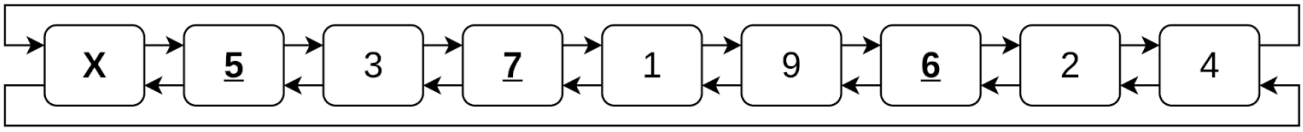
Write a method/function called **rangeMove()**. The function takes three parameters, referencing **one dummy head and two integers**. The method/function will move all the nodes that fall within the given range of integers to the back of the linked list. The method/function will not return anything because the dummy head won't change.

NOTE:

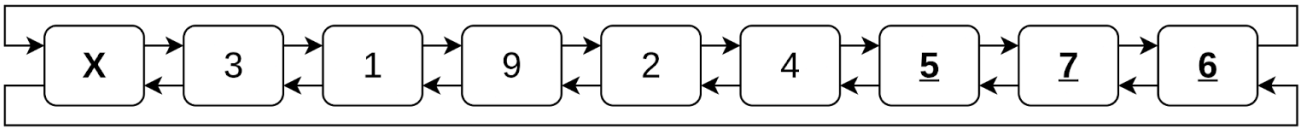
3. YOU **CANNOT** CREATE ANY NEW LINKED LIST FOR THIS TASK
4. YOU **CANNOT** CREATE ANY EXTRA NODE.

Given Dummy Headed Doubly Circular Linked List

Given Range: [start,end] = [5,7]



Expected Modified Linked List



LAB 4 (Part#2)
Secondary Data Structures
[CO1]

Instructions for students:

- You may use Java / Python to complete the tasks.
- If you are using **JAVA**, then follow the **Java Template**.
- If you are using **PYTHON**, then follow the **Python template**.
- **YOU CANNOT USE ANY OTHER EXTRA DATA STRUCTURE, UNLESS IT'S MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

`Python List, Negative indexing, append() or other types of Built-In functions are STRICTLY prohibited`

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 3 Assignment Tasks: 3
Total Marks: 15

Intro Lab Tasks [No Need to Submit]

Implementing an Array-Based Stack

A stack stores elements such that the most recently inserted element is the first one removed. This follows the **Last-In, First-Out (LIFO)** principle. The **ArrayStack** class has only **two private instance variables:**

- An array of Objects called **'stack'** (You take the size using the constructor).
- An integer called **'top'** which initializes at -1. The top index increases when something is "pushed" and it decreases when something is "popped".

Complete the **ArrayStack** class by implementing the following methods:

Methods	Description
push(element)	Inserts an element at the top of the stack. If the stack is full, just print stack overflow. The push always increments the top.
pop()	Removes and returns the top element of the stack. If the stack is empty, just print "stack underflow". The pop() always decrements the top.
peek()	Returns the top element without removing it. If the stack is empty, return null.
isEmpty()	Returns true if the stack contains no elements, otherwise false.

Implementing an Array-Based Queue

A queue stores elements such that the earliest inserted element is the first one removed. This follows the **First-In, First-Out (FIFO)** principle. The **ArrayQueue** class has the following **four private instance variables**:

- An array of Objects called **'queue'** (You take the size using the constructor)
- Three integers:
  - **'front'** – indicates the index of the element at the front of the queue.
  - **'rear'** – indicates the index where the next element will be inserted.
    - [Both front and rear should initialize to 0]
  - **'size'** - indicates the number of elements in the queue

Complete the **ArrayQueue** class by implementing the following methods:

Methods	Description
enqueue(element)	Inserts an element at the rear of the queue. If the queue is full, print "queue overflow". The enqueue always increments the rear. When you reach the end, the value of rear rounds back up to the first index.
dequeue()	Removes and returns the front element of the queue. If the queue is empty, print "queue underflow" (or throw/raise an Exception). The dequeue always increments the front. When you reach the end, the value of rear rounds back up to the first index.
peek()	Returns the front element without removing it. If the queue is empty, return null.
isEmpty()	Returns true if the queue contains no elements, otherwise false.

Lab Tasks [No Need to Submit]

1. Stack: Tower of Blocks

A kid is trying to make a tower using blocks with numbers or characters written on them. He puts a block on the top of another to make the tower. As for removing the blocks, he removes the topmost block if needed. He is afraid that removing other blocks from the middle in one go will result in the collapse of his tower.

Now his friend wants your nth block from the top of your tower. He is willing to give him the nth block from the top, preserving the relative position of others.

Following his tower build process, solve this scenario using **Stack** class. You can only use the stack methods.

In the following examples, consider the rightmost element to be the topmost element of the stack. **(Rightmost item is the top)**

Sample Tower	Sample Output	Explanation
Stack: 4 19 23 17 5 n: 2	Stack: 4 19 23 5	In this tower, the 2nd block from the top is the block with 17. After removing the block the stack looks like 4 19 23 5
Stack: 73 85 15 41 n: 3	Stack: 73 15 41	In this tower, the 3rd block from the top is the block with 85. After removing the block the stack looks like 73 15 41

[NOTE: A LinkedListStack class is already given and you can create an object of Stack to call push(),pop(),peek(),isEmpty() as per your will to complete the task]

You cannot use any data structure other than Stack

2. Queue: Remove Consecutive Duplicates

NOTE: YOU MUST USE QUEUE TO SOLVE THIS TASK

Write a method/function called **removeConsecutiveDups()** that takes only one String in its parameter. You are given a String which may or may not contain consecutive duplicate values. Finally the **removeConsecutiveDups()** method returns a new String containing no consecutive duplicate characters. If no duplicates are found then it simply returns the same string.

Sample Given String	Sample Returned String
aabbbccccdd	abcd
aaabbaa	aba
abcabcabc	abcabcabc
aaaaa	a

[Hint: A LinkedListQueue class containing enQueue(), deQueue(), peek() and isEmpty() methods is already given in the driver code, you just need to create an Object of it to solve the task. No need to design a new Queue]

You cannot use any data structure other than Queue

Assignment Tasks [Need to Submit]

1. Stack: Conditional Reverse

Consider that a Stack class has been implemented containing the push(element), pop(), peek() and isEmpty() functions.

Complete the function **conditional_reverse()** which will take an object of Stack class as an input that contains some integer values. **The function returns a new stack** which will contain the values in reverse order from the given stack **except the consecutive one's.** You cannot use any other data structure except Stack.

In the following example, consider the rightmost element to be the topmost element of the stack.

Sample Input Stack (Rightmost is the top)	**Returned Reversed Stack (Rightmost is the top)**
Stack: 10, 10, 20, 20, 30, 10, 50 **Top** = 50	**Stack:** 50, 10, 30, 20, 10 **Top** = 10
Explanation: Consecutive 20 and 10 are not present in the output reversed stack	

[NOTE: A LinkedListStack class is already given and you can create an object of Stack to call push(),pop(),peek(),isEmpty() as per your will to complete the task]

You cannot use any data structure other than Stack

2. Queue: Hot Potato Game

A group of friends are standing in a circle playing the **Hot Potato game**. The students pass an imaginary "potato" to the next person in the circle. After the potato has been passed a fixed number of times k, the student holding the potato is eliminated from the game. The process then continues with the remaining students, starting from the next student in the circle. This continues until only one student remains, who is declared the winner.

Write a method **hotPotato(players, k)** that simulates this game using a queue. The method takes an array of Strings (players) and an integer (k). The method should repeatedly rotate the queue to simulate passing the potato and remove the player who is eliminated. Finally, the method should **return** the name of the winning player.

Sample Input	Sample Output
players = ["Ali", "Ben", "Cia", "Dan", "Eli", "Faye"] k = 3	Dan eliminated Ben eliminated Ali eliminated Cia eliminated Faye eliminated Winner: Eli
Explanation: **Queue initially:** Ali Ben Cia Dan Eli Faye Passing 3 times → Dan eliminated → Eli Faye Ali Ben Cia Passing 3 times → Ben eliminated → Cia Eli Faye Ali Passing 3 times → Ali eliminated → Cia Eli Faye Passing 3 times → Cia eliminated → Eli Faye Passing 3 times → Faye eliminated → Eli **Remaining player → Eli (Winner)**	
players = ["Ali", "Rafi", "Sara", "Nina"] k = 2	Sara eliminated Rafi eliminated Nina eliminated Winner: Ali

[NOTE: A LinkedListQueue class is already given, you can use it to create an Object of Queue to call enqueue(),dequeue(),peek(),isEmpty() as per your will to complete the task]

You cannot use any data structure other than Queue

3. Stack and Queue: Dancing Pairs

From a dance school, you are given a **Stack filled with Dancer's Info (Objects of Dancer class)**. The Dancer Class only has three instance variables; name(String), gender (char), id (int). The genders are either "M" or "F" (male or female) and the id can be any random integer.

The stack of dancers need to be paired off as male with female for a competition. But this pairing has be done by following the rules of the competition:

- If ***a male & a female*** or ***a female & a male*** are consecutive in the stack, they get paired immediately and popped out of the Stack since they've been processed.
- If two consecutive females are found then the ***first spare female*** should be popped out of Stack and enqueued in a **femaleQueue** for future pairing.
- If two consecutive males are found, we try to pair the ***first spare male*** with a spare female from the **femaleQueue**. If no spare female is available in the then the spare male is enqueued in a **maleQueue**.
- Once all the dancers have been processed from the Stack, the dancers in the **maleQueue** or **femaleQueue** need to be processed, in case they're not empty. If a dancer couldn't be paired up they need to be mentioned at the very end.

Your task is to complete the given method **dance_pair()** that takes **a stack** and prints every male-female pair and those who couldn't be paired as well.

`NOTE: A LinkedListStack & LinkedListQueue class is already given, you can use it to create an Object of Stack to call push(),pop(),peek(),isEmpty() and Queue to call enqueue(),dequeue(),peek(),isEmpty() to complete the task.`

`In both classes, isEmpty() returns true/false, peek() and pop() returns null/None for underflow.`

You cannot use any data structure other than stack and queue

sample examples are given on the next page

Sample Given Stack	Sample Output
Nina(F-12) <- TOP Omar(M-44) Liam(M-24) Sara(F-15) Maya(F-54) Arif(M-10)	#1 Nina(F-12) & Omar(M-44) #2 Liam(M-24) & Sara(F-15) #3 Maya(F-54) & Arif(M-10)
Rita(F-43) <- TOP Mina(F-54) Lina(F-12) Omar(M-9) Arif(M-51) Sami(M-53) Nina(F-29) Lara(F-43) Tariq(M-45)	#1: Lina(F-12) & Omar(M-09) #2: Arif(M-51) & Rita(F-43) #3: Sami(M-53) & Nina(F-29) #4: Lara(F-43) & Tariq(M-45) Unpaired Female: Mina(F-54)
Sara(F-43) <- TOP Lina(F-52) Omar(M-16) Rafi(M-23) Maya(F-65) Alim(M-11) Kazi(M-32) Nina(F-75) Anan(M-11) Wafi(M-13)	#1: Lina(F-52) & Omar(M-16) #2: Rafi(M-23) & Maya(F-65) #3: Alim(M-11) & Sara(F-43) #4: Kazi(M-32) & Nina(F-75) Unpaired Male: Anan(M-11), Wafi(M-13)

LAB 4 (Part#1)
Secondary Data Structures
[CO1]

Instructions for students:

- If you are using **JAVA**, then follow the **Java Template**.
- If you are using **PYTHON**, then follow the **Python Template**.

NOTE:

- **YOU CANNOT USE ANY OTHER EXTRA DATA STRUCTURE, UNLESS IT'S MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

`Python List, Negative indexing, append() or other types of Built-In functions are STRICTLY prohibited`

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 4 [Part#1] Assignment Tasks: 2
Total Marks: 10

Lab Tasks [No Need to Submit]

1. Hashtable: Insert using Forward Chaining:

Write the **insert()** function and **hash_function()** of the HashTable class which uses an array to store a key-value pair, where the key is a string representing a fruit, and the value is an int representing its respective price.

hash_function(key)

Takes a key which is a string, and calculates its length. If length is even, the sum of the ascii values of the even characters is calculated, otherwise, the sum of the ascii values of odd characters is calculated.
Finally, it returns the sum modded by length of the array

If we call __hash_function("apple"):
As len("apple") is odd, characters in odd indexes (p, l) are taken:

sum = ord(p) + ord(l) = 112 + 108 = 220

If the Hashtable object is initialized with length 5, then:

sum % length of Hashtable array = 220 % 5 = 0
So, the function returns 0

insert(key, value)

Creates a node that contains **a *tuple(for python)* or *an array(for java)*** which stores the key-value pair in 0,1 index respectively. Finds index by passing key to hash_function(). If there is no collision, the node is placed in the index.

If there is a collision, forward chaining is applied. Here, the chain should be arranged in **descending order**, i.e.: if the node being inserted has the highest value (price), it will be the head of the chain. Otherwise, you should iterate the chain and insert it in the appropriate position.

[If the same key is given twice for insertion, update the value of that particular key. No need to create new Node]

Assignment Tasks [Need to Submit]

1. Hashtable: Deletion Operation :

You are given the hash function, **h(key) = (key + 3) % 6** for a hash-table of length 6. In this hashing, forward chaining is used for resolving conflict and a 6-length array of singly linked lists is used as the hash-table. In the singly linked list, each node has a next pointer, an Integer key and a string value, for example: (4 (key) , “Rafi” (value)). The hash-table stores this key-value pair.

Implement a function **remove(hashTable, key)** that takes a key and a hash-table as parameters. The function removes the key-value pair from the aforementioned hashtable if such a key-value pair (whose key matches the key passed as argument) exists in the hashtable.

Consider,Node class and hashTable are given to you. You just need to complete the remove(hashTable, key) function.

```
class Node:
  def __init__(self, key, value, next=None):
      self.key, self.value, self.next = key, value, next
```

Sample Input and Output:
Some Key-value pairs:
(34, “Abid”) , (4, “Rafi”), (6, “Karim”), (3, “Chitra”), (22, “Nilu”)

HashTable is given in the following:
0: (3, “Chitra”)
1: (22, “Nilu”) → (4, “Rafi”) → (34, “Abid”)
2: None
3: (6, “Karim”)
4: None
5: None

remove(hashTable, key=4) returns the changed hashTable where (4, “Rafi”) is removed.

New HashTable Output:
0: (3, “Chitra”)
1: (22, “Nilu”) → (34, “Abid”)
2: None
3: (6, “Karim”)
4: None
5: None

remove(hashTable, key=9) returns the same given hashTable since 9 doesn’t exist in the table.

2. Searching in hashtable :

Complete the **`hashFunction()`** and **`searchHashtable()`** methods. **Do not** change the given code; implement only the required methods. Creating and Inserting into a hash table using forward chaining is already done inside of the class. Do not initialize any other instance variable other than the given ones.

a. **search_hashtable()** → this instance method takes a **key value pair** in the parameter then uses the string key to search for the string in the instance variable **ht**. If the **`string s`** is found, this method returns **'Found'**, else returns **'Not Found'**.
[Do not implement sequential search, implement the hash based search]

b. **hashFunction()** → this instance method takes a key-value pair (string, int), calculates the hashed index on key and returns the index. This hash function takes consecutive two letters of the key string, concatenates their ascii values into an integer and sums all the concatenated integers. Then it finds out the modulus of the summation (**think for yourself with which number should we mod the summation**) as the hashed index.
For instance, for a string 'Mortis', the consecutive two letters are Mo, rt, is.
The concatenated integer for
Mo is 77111 (Ascii of M is 77, o is 111);
rt is 114116 (Ascii of r is 114, t is 116);
'is' is 105115 (Ascii of i is 105, s is 115).
The summation is = 77111+114116+105115
Mod the summation with ___ and return the answer as the hashed index.
[**`fill in the ___ gap with your knowledge`**]

Note: For an odd length string, add the letter 'N' at the end of it. Thus, 'Morti' becomes 'MortiN' and the consecutive two letters are Mo, rt, iN.

LAB 5 Part 1 : Binary Tree

[CO2]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task just follow the given template.
- If you are using **JAVA**, then follow the **Java Template**.
- If you are using **PYTHON**, then follow the **Python Template**.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY UNLESS MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

For Python: List, Negative indexing and append()
and
For Java: Static or Instance variables
are STRICTLY prohibited

Lab Tasks: 1-5

Assignment Tasks: 6-8 (3 tasks)

Total Assignment Mark: 3*5=15

Lab Task 0: Basics of Binary Tree

This is an intro task & there isn't any driver code for this

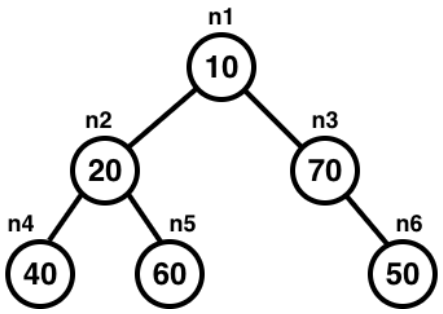
- ❖ For java, create a **separate folder** for this task and follow the instructions.
- ❖ For Python, create a **separate colab/ipynb/py** file and follow the instructions.

➔ Design a TreeNode class.

- Declare three instance variables, one called **elem** (Object data type for java), one called **left** (TreeNode datatype) and another called **right** (TreeNode datatype).
- Write a constructor that has only one parameter (Object type for Java) and assigns the parameter value to the **elem** instance variable.

➔ Design a Tester class (for java) / Design another code cell (for python)

- Create 6 different objects of TreeNode class. Assign values as shown in the illustration.
- Variable names should be: **n1, n2, n3, n4, n5, n6**.
- Connect the 6 TreeNodes as shown in the illustration which will form a binary tree. Here **n1** will be the root of the tree.



Now, execute the lines given below and try to understand the output. You may need pen & paper. If there are errors, try to figure out why that error occurred and how to fix it.

Note: Java & Python output might not always be the same.

JAVA	PYTHON
`System.out.println( n1.left );`	`print( n1.left )`
`System.out.println( n3.right.elem );`	`print( n3.right.elem )`
`TreeNode x = n2.left;` `System.out.println( n1.elem + x.elem );`	`x = n2.left` `print( x.elem + n6.elem )`

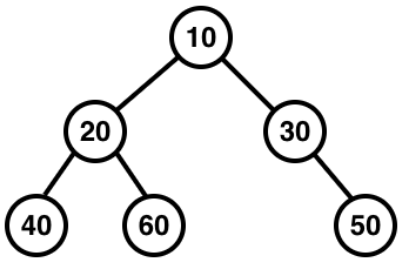
`x = new TreeNode(80);`
`n3.left = x;`
`System.out.println(n1.right.left.elem);`
	`x = TreeNode(80)`
	`n3.left = x`
	`print(n1.right.left.elem)`

`System.out.println(n1.left.right + n5.left);`	`print(n1.left.right + n5.left)`
`n1.left.right = null;` `System.out.println(n1.left.right.elem)`	`n1.left.right = None;` `print(n1.left.right.elem)`

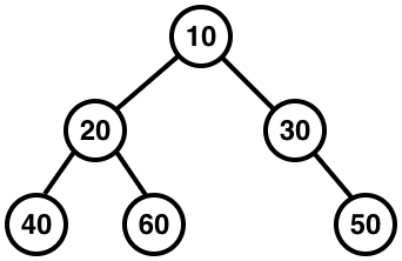
LAB TASKS

1. Basic Traversal:

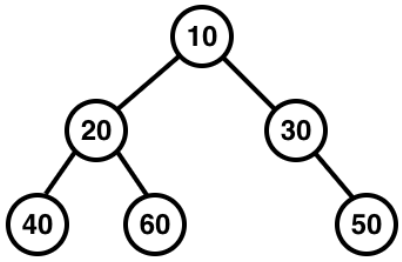
1.1. Given the **root** of a binary tree, print the tree **pre-order**.

Sample Given Tree	Sample Output
	10 20 40 60 30 50

1.2. Given the **root** of a binary tree, print the tree **post-order** and the levels of each node. The level of the root is 0. You can use a helper.

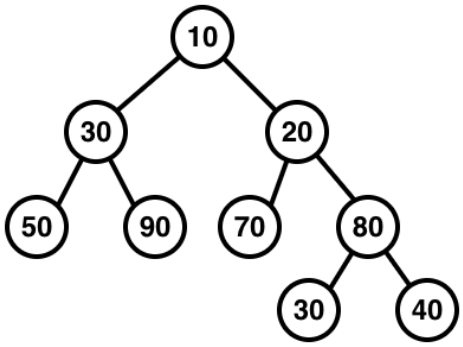
Sample Given Tree	Sample Output
	40:2 60:2 20:1 50:2 30:1 10:0

1.3. Given the **root** of a binary tree, print the tree **in-order** and the levels of each node only **if the levels are even**. The level of the root is 0.

Sample Given Tree	Sample Output
	40:2 60:2 10:0 50:2

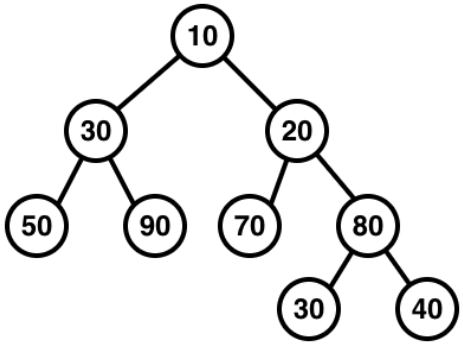
The 3 tasks above should give you a general idea about how traversing a tree works. Try to understand why the root was printed at the very beginning in 1.1 and at the very end in 1.2. You can simulate the recursive calls using pen and paper.

2. Given the **root** of a binary tree, **return** the **total number of nodes** in that tree.

Sample Given Tree	Sample Returned Value
	9

3. Summation of the tree:

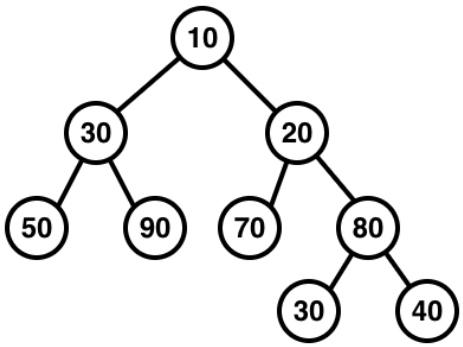
3.1. Given the **root** of a binary tree, **return** the summation of nodes in that tree. (You **won't** need helper function)

Sample Given Tree	Sample Output
	420

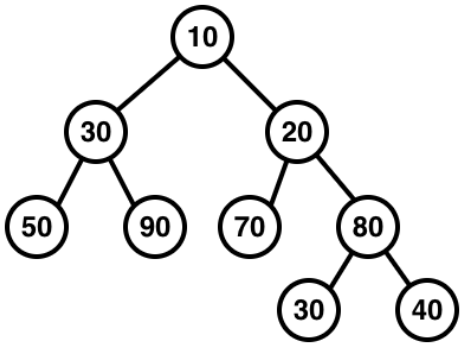
3.2. Given the **root** of a binary tree, **print** the summation of nodes in that tree. (think how to re-use 3.1 and try to understand the difference)

Sample Given Tree	Sample Returned Value
Same as above sample	Same as above sample

4. Given the **root** of a binary tree, print all the **leaf** nodes of the tree.

Sample Given Tree	Sample Output
	`50 90 70 30 40`

5. Given the **root** of a binary tree, return the max of all nodes.

Sample Given Tree	Sample Returned Value
	`90`

ASSIGNMENT TASKS

6. Subtraction of Nodes:

Write a **recursive** function **subtract_summation**() that takes the root of a binary tree as a parameter. The function will **subtract** the **summation** of the **right subtree** of the given root **from** the **summation** of the **left subtree** of the given root. Consider, the **Node** class for Binary Tree already defined with elem, left and right variables. You can use helper functions.

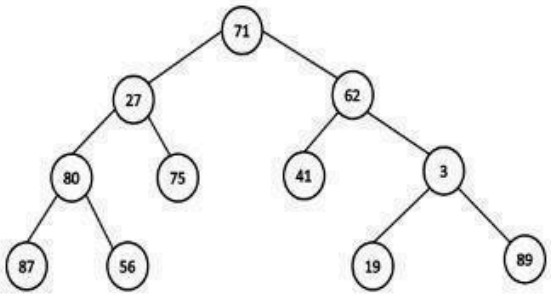
YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function.

Python Notation:

```
def subtract_summation(root):
    // To do
    return None
```

Function Call :

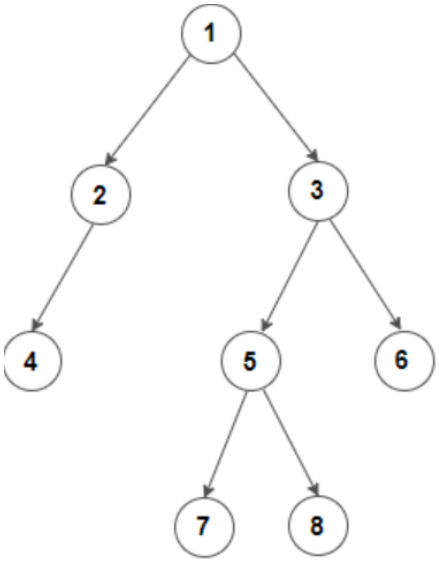
print(**subtract_summation**(root)). Here root refers to the tree below.

Sample Input	Sample Output	Explanation
	111	Summation of left subtree - summation of right subtree = (27+75+80+87+56) - (62+41+3+19+89) = 111

more tasks on the next page>>

7. Difference of Level Sum**:**

Given a Binary Tree, Write a function that finds the difference between sum of all nodes present at odd and even levels in a binary tree, i.e. sum of all odd level nodes - sum of all even level nodes.

Sample Input:	**Sample Output**	**Explanation**
	4	-1+2+3-4-5-6+7+8 = 4

more task on the next page>>

8. Swap Children Nodes:

Write a **recursive** function **swap_child()** that takes the root of a binary tree, node's level and a number M as a parameter. The function will swap the left and right children of all the nodes at level M and above. Here, 0 < M < height of the tree (root's height). Consider, the Node class for Binary Tree already defined with elem, left and right variables. **YOU CANNOT USE LIST OR DICTIONARY, any built-in function, global variables.**

Python Notation:

```
def swap_child(root, level, M):
        # To do
```

Function Call :

swap_child(root, 0, 2). Here root refers to the tree below.

Input Tree	Resulting Tree	Explanation
		Here M = 2 and all the nodes from level 2 and above are swapped left with right. Here above means the level that situated at a higher position of the tree

LAB 5 Part2 : Binary Tree

[CO2]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task just follow the given template.
- If you are using **JAVA**, then follow the **Java Template**.
- If you are using **PYTHON**, then follow the **Python Template**.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY UNLESS MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

For Python: List, Negative indexing and append()
and
For Java: Static or Instance variables
are STRICTLY prohibited

Lab Tasks: 3 tasks (1~3)

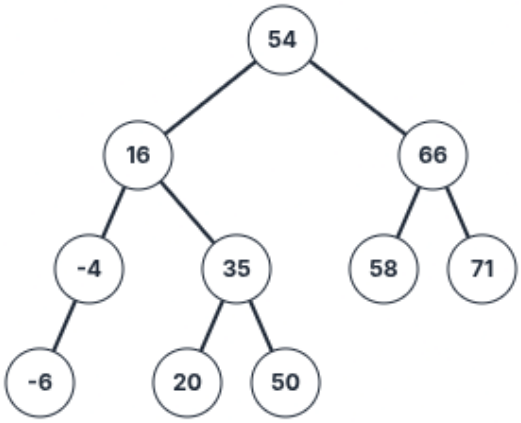
Assignment Tasks: 3 tasks (4~6)

Total Assignment Mark: 3*5=15

LAB TASKS (no need to submit)

1. Binary Search Tree Minimum and Predecessor:

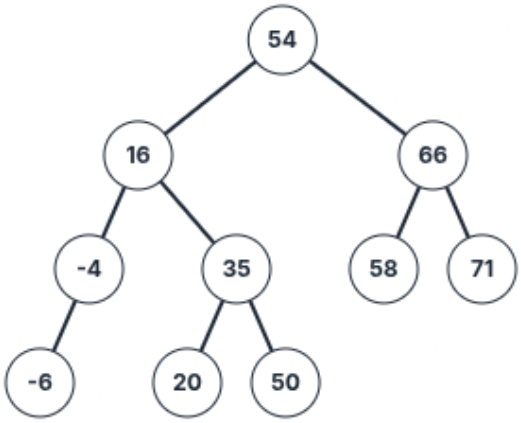
a. Find the maximum number from a Binary Search Tree



Sample Call	Result
`getMaximum(root)`	71

Note: You have to keep in mind that, the question says it's a Binary Search Tree (not a regular BT). Therefore, you must use BST logic to find the maximum.

b. Find a InOrder Predecessor from a Binary Search Tree



Sample Call	Result
`inOrderPred(root,54)`	50
`inOrderPred(root,66)`	58
`inOrderPred(root,20)`	16
`inOrderPred(root,50)`	35
`inOrderPred(root,-6)`	null/None

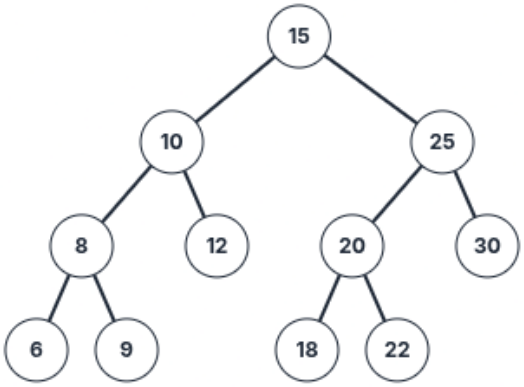
Note: You should reuse the lab task 1a to solve 1b.

You can solve 1a and 1b without using any recursion. Can you guess why we can solve them without recursion even though they are binary tree problems?

2. Find the Lowest Common Ancestor (LCA) of two nodes in a BST

The lowest common ancestor (LCA) of two nodes x and y in the BST is the lowest (i.e., deepest) node that has both x and y as descendants. In other words, the LCA of x and y is the shared ancestor of x and y that is located farthest from the root.

Corner Case: if y lies in the subtree rooted at node x, then x is the LCA; otherwise, if x lies in the subtree rooted at node y, then y is the LCA.



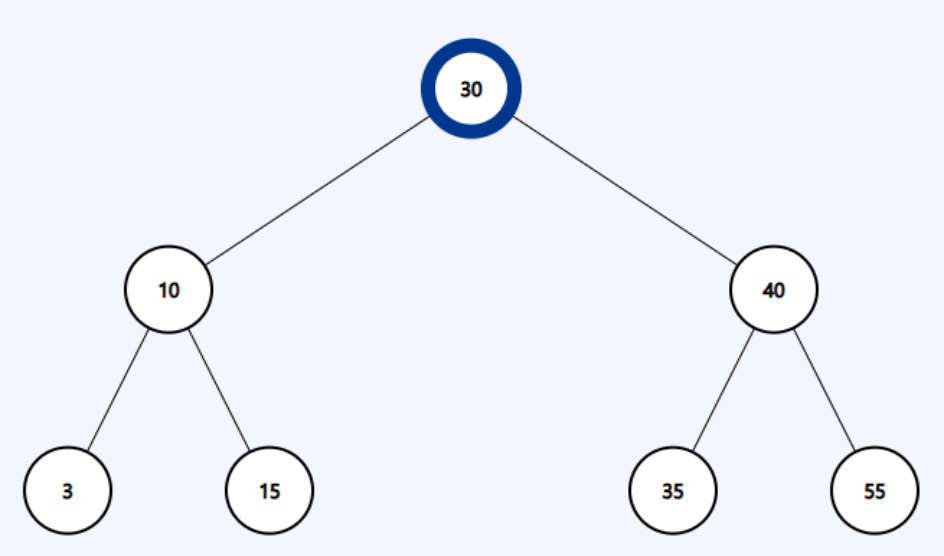
Sample Call	Result
LCA(root, 6,12)	10
LCA(root, 20,6)	15
LCA(root, 18,22)	20
LCA(root, 20,25)	25
LCA(root, 10,12)	10

3. Find the path

Faria studies in Rangpur Medical College. During her Eid vacation, she is planning to visit her family and relatives in Chittagong. But she feels really bad during the journey. As a result, you need to help Faria reach her destination.

A Binary Search Tree is given.The root node is the starting position of Faria and a key node will be given as the destination. Now, you have to find if the following key node (which is the destination of Faria) is present in the tree or not. If present, return the path from the root node to the key node else print “No Path Found”.

[You can use Python List for this task]

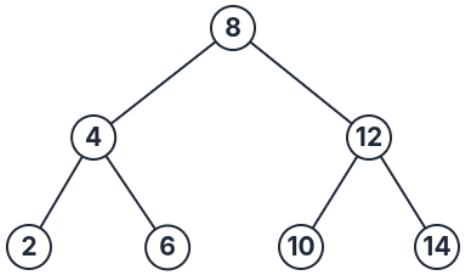
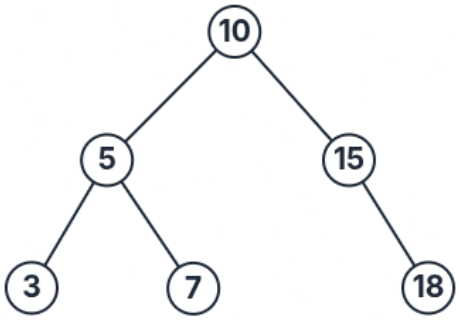


Sample Input	Sample Output
Source node(root): 30 Destination node(key): 15	Path: [30, 10, 15]
Source node(root): 30 Destination node(key): 50	No Path Found

ASSIGNMENT TASKS (must submit)

4. Sum of range

Write a function **rangeSum(root,low,high)** that takes the root of a Binary Search Tree as a parameter and sums up the values within the range mentioned in the parameter.

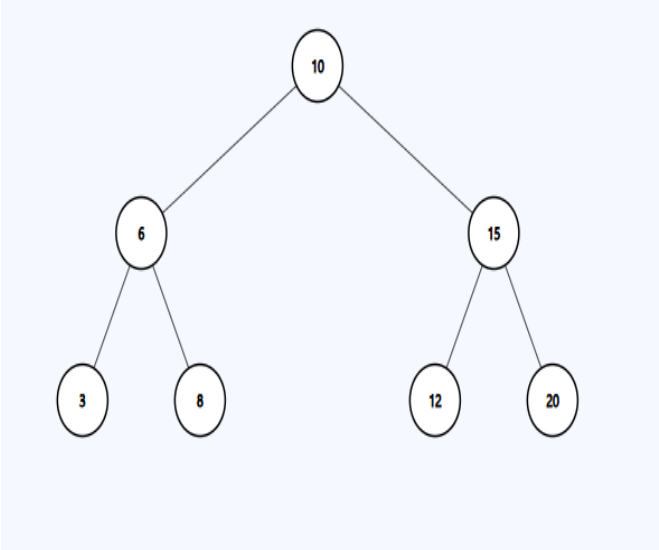
<table>
<tr><th>Sample Input</th><th>Sample Function Call</th><th>Sample Returned Value</th></tr>
<tr><td rowspan="2"> 
</td><td rowspan="2"><code>rangeSum(root, 5, 13)</code></td><td>36</td></tr>
<tr><td>Explanation:
6+8+10+12=36</td></tr>
<tr><td rowspan="2"> 
</td><td rowspan="2"><code>rangeSum(root, 7, 15)</code></td><td>32</td></tr>
<tr><td>Explanation:
7+10+15=32</td></tr>
</table>

5. Mirror Sum of a BST

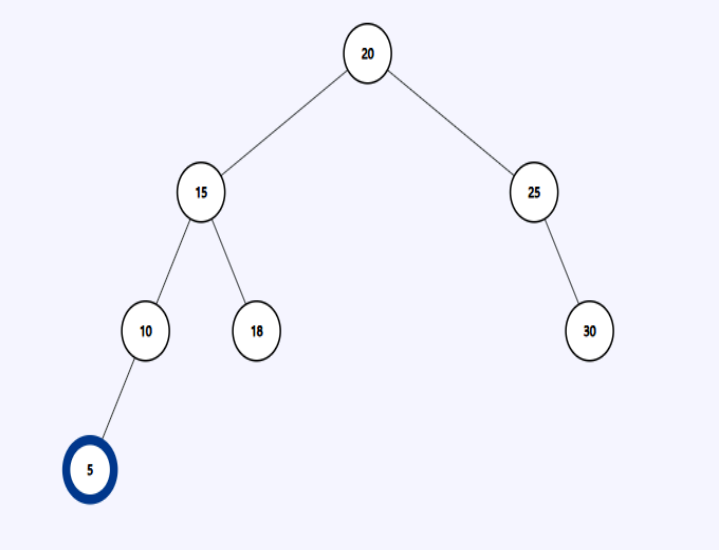
You are given the root node of a Binary Search Tree. You need to calculate the **sum** of the values of the nodes that are **mirrors** of each other then **return** it. Here, mirror means the nodes that are located in corresponding positions in the left and right subtrees of the root. If the mirror doesn't exist then do not sum.

Note: You can use helper functions.

Example Tree input 1



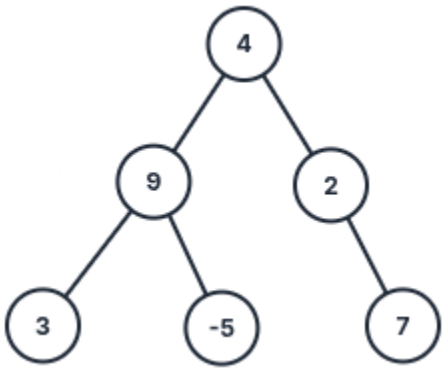
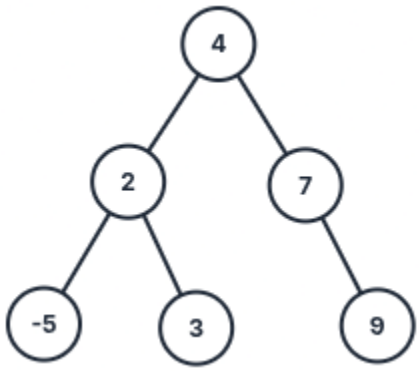
Example Tree input 2



Sample Input 1	Sample Output 1	Explanation 1
mirrorSum(root)	64	For Tree 1 Mirror nodes are: 6 and 15, sum = 6 + 15 = 21 3 and 20, sum = 3 + 20 = 23 8 and 12, sum = 8 + 12 = 20 Total Mirror Node Sum = 21+23+20 = 64
Sample Input 2	**Sample Output 2**	**Explanation 2**
mirrorSum(root)	80	For Tree 2 Mirror nodes are: 15 and 25, sum = 15 + 25 = 40 10 and 30, sum = 10 +30 = 40 Total Mirror Node Sum = 40 + 40 = 80

6. Check BST

Write a function **isBST(root)** that takes the root of a Binary Tree as a parameter and then returns true if its a BST otherwise returns false.

Sample Input	Sample Returned Value
	**false**
	**true**

LAB 6 : Heap

[CO2]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

Python List, Negative indexing and append() is STRICTLY prohibited

Lab Tasks: 3 tasks (1~3)
Assignment Tasks: 3 tasks (4~6)

Total Assignment Mark: 3*5=15

You were given instructions and driver code for the last 6 lab files. However, for this lab, only Task#1 driver codes will be given. Write the other driver codes yourself in your preferred language (Java / Python).

LAB TASKS

Task1: Implementing an Array-Based Min Heap

Driver Codes for Task#1 only [Java Task#1 Template] & [Python Task#1 Template]

A **Min Heap** is a specialized binary tree where the value of each node is **less than or equal to its children**. This ensures that the **smallest element is always at the root**.

In this implementation, the heap is represented using an **array**, where the tree structure is maintained through 1-based index relationships.

The MinHeap class contains the following private instance variables:

An Integer array **heap**	stores the elements of the heap
An integer **size**	represents the current number of elements in the heap
An integer **capacity**	represents the maximum number of elements the heap can hold

Instance Methods to Implement	Steps
insert(value) Description: Inserts a new value into the heap while maintaining the heap property.	1. Check if the heap is full (if **size** is greater or equal to the **capacity**) → If full, print "Heap is full" and do not insert 2. Insert the value at the end of the array 3. Restore heap property using **heapifyUp**
extractMin() Description: Removes and returns the smallest element (root of the heap).	1. If the heap is empty, return null 2. Store the root value ***heap[1]*** (this will be returned) 3. Move the last element to the root position 4. Decrease the size of the heap 5. Restore heap property using **heapifyDown** 6. Return the stored minimum value
peek() Description: Returns the smallest element without removing it.	1. If the heap array is empty, return ***null*** 2. Return the root element ***(heap[1])***
heapifyUp(index) Description: A private method that restores the heap property by moving an element upward. **When used:** After insertion	1. Compare the current element with its parent 2. If the parent is greater, swap them 3. Continue until the heap property is satisfied or root is reached

heapifyDown(index) Description: A private method that restores the heap property by moving an element downward. **When used:** After extracting the minimum element	1. Compare the current element with its children 2. Swap with the smallest child if needed 3. Continue until the heap property is satisfied or reached leaf
isEmpty()	Return **True** if the heap size is 0, otherwise return **False**

Task 2:

Complete task 1 for MaxHeap.

Note:
For MaxHeap class, the instance variables & instance methods would be the same as MinHeap except for extractMin(), instead it'll be named extractMax().

The MaxHeap sorts the element in ascending order.

The instance variables must be Private.

Task 3:

Add a ***static method*** to the class from **Task#1** or **Task#2** (depends on whether you want to use MaxHeap or MinHeap) that performs **out-of-place heap sort** using a heap (min-heap or max-heap). The method will be called ***outPlaceHeapsort()*** and it takes an array in its parameter. The method will do the following:

- Create a Min-Heap or Max-Heap instance.
- Loop through the given array:
    - Insert each element into the heap.
- Loop until the Max-Heap or Min-Heap is empty:
    - Extract the minimum/maximum element from the minheap/maxheap.
    - put them in their sorted order in the original array

Note: The time complexity of the solution is going to be **`n*log(n)`**.

You can utilize the Classes you created for the Lab Task by creating their objects to complete the Assignment Tasks. Just like Stack & Queue tasks, you'll create MaxHeap/MinHeap objects and call their methods. You cannot access the instance variables from outside since they're private.

ASSIGNMENT TASKS

Task 4 [5 marks] :

You are given an array of n tasks, where the value of index i represents the processing time for i-th task. You are also given m machines, each capable of processing one task at a time. Your goal is to distribute the tasks among the machines such that the overall completion time for each machine is minimized. Represent the output as an array where each index corresponds to a machine, and the value at each index is the total processing time of tasks assigned to that machine.

Input Format:

- **tasks:** An array of integers where each integer represents the processing time of a task.
- **m:** An integer representing the number of machines.

Output Format:

- An array of integers of size m, where the value at index i represents the total processing time of tasks assigned to the i-th machine.

Sample Given Input:
tasks = [2, 4, 7, 1, 6]
m = 4

Sample Output:
[2, 4, 7, 7]

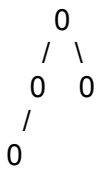
Explanation:
To simplify the use of a heap, we focus only on the current load of each machine (not its index). The min-heap will store the current loads of the machines. Initially, all machines will have a load of 0.
Once all tasks are assigned, the heap will contain the loads of all machines. The array of this heap will represent the output.

Steps to Solve:

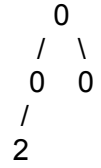
- ➔ Initialize a min-heap object with m length, all set to 0 (representing the load of each machine).
- ➔ For each task, do the following:
  - Extract the smallest load from the heap.
  - Add the task's processing time to this load.
  - Reinsert the updated load into the heap.
- ➔ The final array representation for your heap is the answer.

Example Walkthrough:

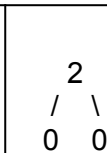
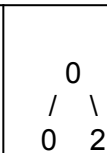
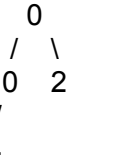
Initial Heap:

Heap Array	[0, 0, 0, 0]
Heap Visual	

Task 1 (Processing Time = 2): Assign to the machine with load 0 (current minimum load).

	Extract the smallest	Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 0, 0, None/null]	0+2=2 is inserted After insertion: [0, 0, 0, 2]
Heap Visual		

Task 2 (Processing Time = 4): Assign to the machine with load 0 (current minimum load).

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 0, 2, None/null]			0+4=4 is inserted After insertion: [0, 0, 2, 4]
Heap Visual				

Task 3 (Processing Time = 7): Assign to the machine with load 0 (current minimum load)

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	0 extracted After extraction: [0, 4, 2, None/null]			0+7=7 is inserted After insertion: [0, 4, 2, 7]
Heap Visual	0 / \ 0 2 / 4	4 / \ 0 2	0 / \ 4 2	0 / \ 4 2 / 7

Task 4 (Processing Time = 1): Assign to the machine with load 0 (current minimum load).

	Extract the smallest			Add the extracted with current & reinsert		
Heap Array	0 extracted After extraction: [0, 4, 2, None/null]			0+1=1 is inserted After insertion: [1, 2, 7, 4]		
Heap Visual	0 / \ 4 2 / 7	7 / \ 4 2	2 / \ 4 7	2 / \ 4 7 / 1	2 / \ 1 7 / 4	1 / \ 2 7 / 4

Task 5 (Processing Time = 6): Assign to the machine with the smallest load, that is 1, and add the current processing time (6) with this.

	Extract the smallest			Add the extracted with current & reinsert
Heap Array	1 extracted After extraction: [2, 4, 7, None/null]			1+6=7 is inserted After insertion: [2, 4, 7, 7]
Heap Visual	1 / \ 2 7 / 4	4 / \ 2 7	2 / \ 4 7	2 / \ 4 7 / 7

Final Output:
Finally, the array representation of the heap is [2, 4, 7, 7]

Hints:

1. Tasks must be assigned one at a time to the machine with the current smallest load.
2. Use a heap (you have built before) to efficiently find the machine with the smallest load.
3. You can create a function that will receive tasks and m and return the final heap for simplicity

Task 5 [5 marks] :

You are given an array of integers and a number k. Your task is to find the top k largest elements from the list. Use a max-heap to solve this problem efficiently.

Input Format:

- nums: An array of integers.
- k: An integer representing the number of largest elements to find.

Output Format:

- An array of k integers representing the largest elements in descending order.

Sample Input:
nums = [4, 10, 2, 8, 6, 7]
k = 3
Output:
[10, 8, 7]

Explanation:

To solve this, we use a max-heap because it allows efficient retrieval of the largest elements. The process involves:

1. Insert all numbers from the input list into a max-heap.
2. Create a new array with size k.
3. Extract the maximum k times and store it to the new array to get the top k largest elements.
4. Return the result array in descending order.

Steps to Solve:

1. Build a max-heap where the length of the array will be equal to length of nums.
2. Extract the maximum k times and store the values in the newly created result array.
3. Return the result array.

Hints:

1. You must use a max-heap **(created above in Task2)** to solve the problem.
2. k is always smaller than or equal to the size of the array.
3. You can create a function that will receive the nums array and k for simplicity.

For TASK#6 You need build a new custom MaxHeap

Task 6 [5 marks] : Task Scheduler with Priority Queue

You are given two arrays: one containing task names and another containing their corresponding priorities. Your task is to process all tasks in order of their priority using a Max-heap. Tasks with higher priority values should be processed first.

Input Format:

- *task_names:* An array of strings where each element represents a task name.
- *priorities:* An array of integers where each element represents the priority of the corresponding task.

Output Format:

- An array of strings representing the task names in the order they should be processed (highest priority first).

Steps to Solve:

1. Build a customized max-heap where each element contains both the task name and its priority.
2. Iterate through both arrays simultaneously and insert each (task_name, priority) pair into the max-heap (heap should compare based on priority).
3. Extract the maximum repeatedly until the heap is empty, storing each task name in the result array.
4. Return the result array containing all task names in priority order.

Sample Input
task_names = ["Email", "Meeting", "Code Review", "Lunch", "Debug"] priorities = [2, 5, 3, 1, 4]
Sample Returned Priorities Array
["Meeting", "Debug", "Code Review", "Email", "Lunch"]
Explanation
To solve this problem, we use a max-heap like a priority queue because it allows efficient retrieval of the highest priority task. The process involves: 1. Combine task names with their priorities and insert all tasks into a max-heap based on their priority values. 2. Create a result array to store task names in processing order. 3. Extract the maximum (highest priority task) repeatedly until the heap is empty. 4. Return the result array containing task names in descending order of priority.

LAB 7 : Graph

[CO5]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY UNLESS MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

Python List, Negative indexing and append() is STRICTLY prohibited unless mentioned in the question

Lab Tasks: 4 tasks (0~1b)

Assignment Tasks: 6 tasks (2a~4b)

Total Assignment Mark: 6*5=30

Dear Students, you have been given instructions and driver code for the majority of the labs. For this lab no driver code will be given. You will develop everything (necessary functions, class, driver code, etc.) in your preferred language (Java or Python).

To solve the problems, draw a graph (**https://graphonline.top**) according to your preference. However, your graph must have a minimum of **7 vertices** and a minimum of **12 edges**. You can bring the graph you drew on the website to your code by going to Graph menu -> Adjacency Matrix menu and copying the matrix.

Note: Solve all the problems for both adjacency matrix representation and adjacency list representation. You can write different functions for each task to make your life easier.

Task 0: [Lab Task]

Represent the Graph you just drew in your code. Using both **Adjacency Matrix (Task0a)** and **Adjacency List (Task0b)**.

Task 1: [Lab Task]

Given an undirected, unweighted graph as input, write a function to find the vertex with maximum degree and return the degree of that vertex. Solve it for Adjacency Matrix **(Task1a)** & Adjacency List **(Task1b)**.

Task 2: [Assignment Task]

Given an undirected, edge-weighted graph as input, write a function to find the vertex whose sum of edge weights is maximum. Solve it for Adjacency Matrix **(Task2a)** & Adjacency List **(Task2b)**.

Task 3: [Assignment Task]

Solve Task 1 and Task 2 for directed, edge-weighted graphs considering only outgoing edges. Solve it for Adjacency Matrix **(Task3a)** & Adjacency List **(Task3b)**.

Task 4: [Assignment Task]

Write a function that will receive a directed weighted graph and convert it to an undirected weighted graph. Consider the weight of the edges will be unchanged. If there's a case where you have a bidirectional edge then you can sum up their weights. Solve it for Adjacency Matrix **(Task4a)** & Adjacency List **(Task4b)**.

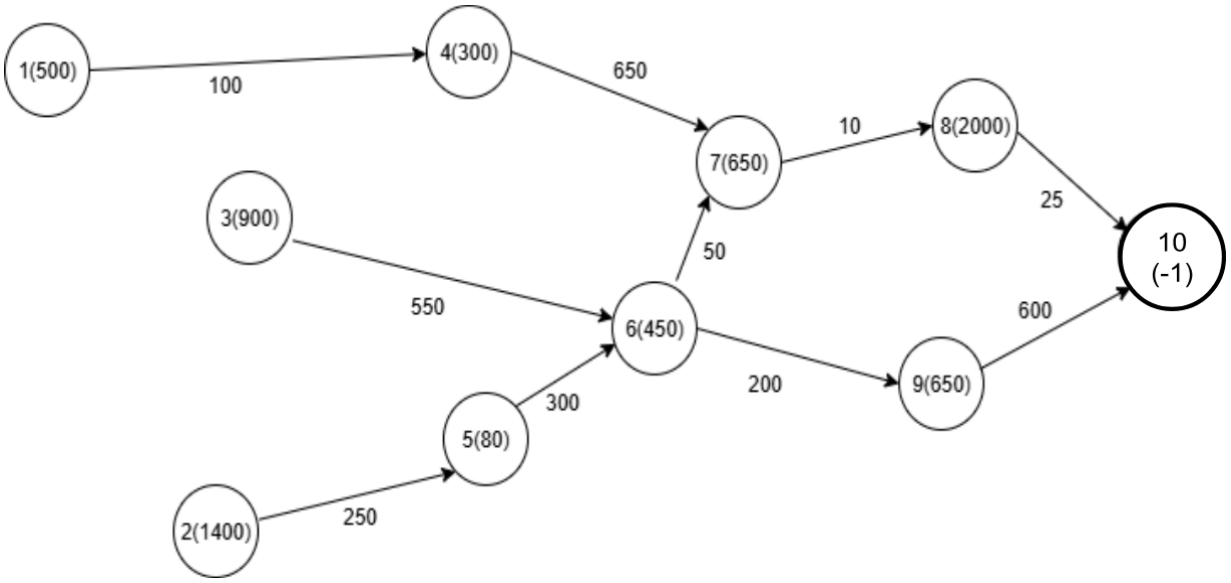
UNGRADED CODING TASK [Java Template,Python Template]

[Note: Try solving this coding question using Adjacency Matrix as well]

Luffy and Shanks start from given islands with initial power and aim to reach **Laugh Tale (vertex warLord power value is -1)**. Between islands, they fight a **Marine Officer (power given as edge weight)**. On each island, they fight a **Warlord (power given as vertex value)**. They always choose the next island with the **minimum (Marine Officer + Warlord) power**. Furthermore, stamina decreases after every single fight. If a pirate's stamina becomes less than the Warlord's power at any island, they cannot continue and fail.

Winner rules:

- If only one of them reaches Laugh Tale and the other can't → the one who reaches wins.
- If both reach Laugh Tale → the one with higher remaining stamina wins.
- If none reach Laugh Tale → Winner is no one.



[You can assume there's won't be any cyclic paths from the starting point to Laugh Tale]

[You can write helper functions if needed]

Python Method Signature
`def laughTale(wL, adjL, sSt, sP, lSt, lP):` `  #TODO`
Java Method Signature

`public static void laughTale(Integer[] warLords, Edge[] adjL, int sSt, int sP, int lSt, int lP){`
`  //TODO`
`}`

Warlord Powers, Sample Graph & Edge Class

warLords powers : [null, 500, 1400, 900, 300, 80, 450, 650, 2000, 650, -1]

Given Adjacency List :

0 : null
1 : <4,100> -> null
2 : <5,250> -> null
3 : <6,550> -> null
4 : <7,650> -> null
5 : <6,300> -> null
6 : <7,50> -> <9,200> -> null
7 : <8,10> -> null
8 : <10,25> -> null
9 : <10,600> -> null
10 : null

Given Java Edge Class

```
class Edge{
       int toV, weight;
       Edge next;
       public Edge(int t, int w){
              this.toV = t;
              this.weight = w;
       }
}
```

Given Python Edge Class

```
class Edge:
    def __init__(self, toV, weight):
        self.toV = toV
        self.weight = weight
        self.next = None
```

Sample Starting Vertex and Graph	Output
sSt = 2, sP = 7000, lSt = 1, lP = 3000	Winner is Shanks

Explanation

sSt: Shank's start from Island: 2
sP: Shanks's power: 7000
lSt: Luffy starts from Island: 1
lP: Luffy's power: 3000

Luffy's path: vertex 1 -> vertex 4 -> vertex 7 -> vertex 8
Luffy's power level: 3000-500-100-300-650-650-10=790 *[no more power left to reach vertex 10]*
Shanks's path: vertex 2 -> vertex 5 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10
Shanks's power level: 7000-1400-250-80-300-450-50-650-10-2000-25= 1785.
So, Luffy cannot reach Laugh Tale and the **Winner is Shanks.**

Sample Starting Vertex and Graph	Output
sSt = 2, sP = 7000, lSt = 3, lP = 6500	Winner is Luffy

sSt: Shank's start from Island: 2
sP: Shanks's power: 7000
lSt: Luffy starts from Island: 3
lP: Luffy's power: 6500

Luffy's path: vertex 3 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10
Luffy's power level: 6500-900-550-450-50-650-10-2000-25=1865

Shanks’s path: vertex 2 -> vertex 5 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10
Shanks’s power level: 7000-1400-250-80-300-450-50-650-10-2000-25= 1785
Since 1865>1785. So, the **winner is Luffy**.